

Application of BFS/DFS

Shortest Path Algorithms and Tree Diameter

Amaan Khan

ANCC CAIC Lecture Series

18 June 2026

Recap: Traversal

- We know how to traverse graphs: There are mainly two ways DFS which explores a path fully once before moving on to another and BFS which explores layer by layer.

Recap: Traversal

- We know how to traverse graphs: There are mainly two ways DFS which explores a path fully once before moving on to another and BFS which explores layer by layer.
- Today, we will look at more important applications of both DFS and BFS.

Recap: Traversal

- We know how to traverse graphs: There are mainly two ways DFS which explores a path fully once before moving on to another and BFS which explores layer by layer.
- Today, we will look at more important applications of both DFS and BFS.

The Core Question

What if we want to find the **shortest** path between two vertices of a graph?

Recap: Traversal Algorithms

Depth-First Search (DFS)

```
function dfs(v) {  
    if v is explored {  
        return  
    }  
    mark v explored  
    for each unexplored  
    neighbour w of v {  
        dfs(w)  
    }  
}
```

Breadth-First Search (BFS)

```
function bfs(u) {  
    Create queue q  
    mark u explored  
    q.enqueue(u)  
    while q not empty {  
        v = q.dequeue()  
        for each unexplored  
        neighbour w of v {  
            mark w explored  
            q.enqueue(w)  
        }  
    }  
}
```

Unweighted Shortest Paths (BFS)

- Look at it as if all edges had the same weight.

Unweighted Shortest Paths (BFS)

- Look at it as if all edges had the same weight.
- BFS is perfectly designed for this task because it searches layer-by-layer:

Unweighted Shortest Paths (BFS)

- Look at it as if all edges had the same weight.
- BFS is perfectly designed for this task because it searches layer-by-layer:
 - Layer 1: All nodes 1 edge away.

Unweighted Shortest Paths (BFS)

- Look at it as if all edges had the same weight.
- BFS is perfectly designed for this task because it searches layer-by-layer:
 - Layer 1: All nodes 1 edge away.
 - Layer 2: All nodes 2 edges away.

Unweighted Shortest Paths (BFS)

- Look at it as if all edges had the same weight.
- BFS is perfectly designed for this task because it searches layer-by-layer:
 - Layer 1: All nodes 1 edge away.
 - Layer 2: All nodes 2 edges away.
 - Layer k : All nodes k edges away.

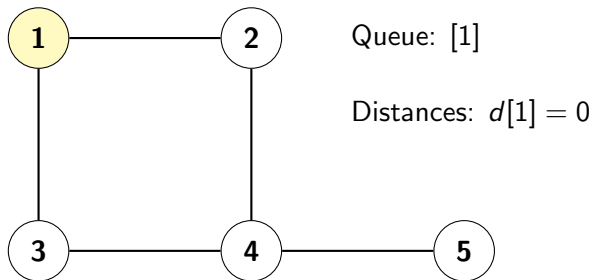
Unweighted Shortest Paths (BFS)

- Look at it as if all edges had the same weight.
- BFS is perfectly designed for this task because it searches layer-by-layer:
 - Layer 1: All nodes 1 edge away.
 - Layer 2: All nodes 2 edges away.
 - Layer k : All nodes k edges away.

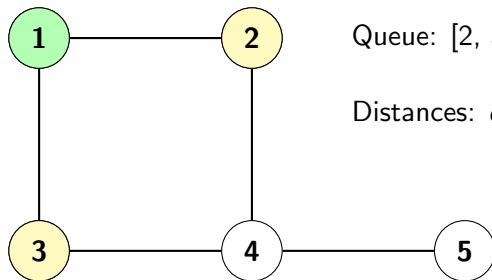
The BFS Guarantee

The first time we reach a node during a BFS, the path taken is guaranteed to be the shortest path.

BFS Dry Run



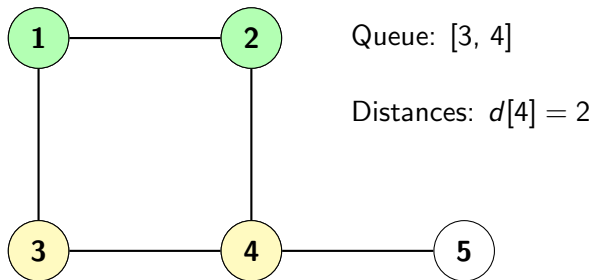
BFS Dry Run



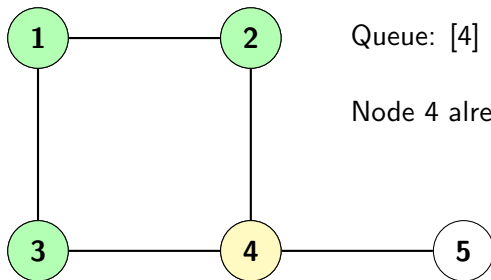
Queue: [2, 3]

Distances: $d[1] = 0, d[2] = 1, d[3] = 1$

BFS Dry Run



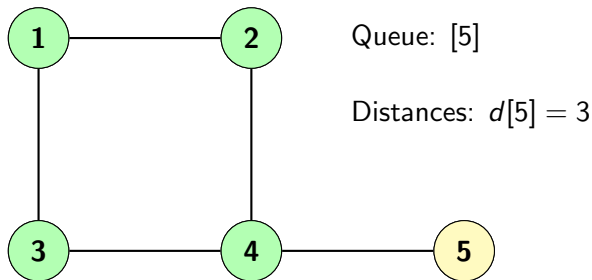
BFS Dry Run



Queue: [4]

Node 4 already visited. Skip.

BFS Dry Run



Introducing Edge Weights

- What if now the edges had weights associated to them.

Introducing Edge Weights

- What if now the edges had weights associated to them.
- BFS blindly assumes all edges cost exactly the same.

Introducing Edge Weights

- What if now the edges had weights associated to them.
- BFS blindly assumes all edges cost exactly the same.
- We need new algorithms to calculate the shortest path when costs vary!

Introducing Edge Weights

- What if now the edges had weights associated to them.
- BFS blindly assumes all edges cost exactly the same.
- We need new algorithms to calculate the shortest path when costs vary!

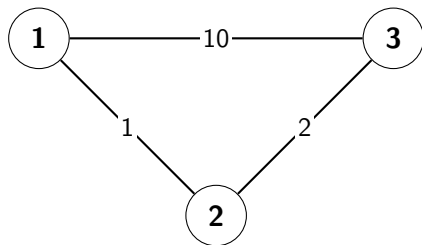
Updating Distances

The core operation for weighted graphs is **checking for better paths**. If traversing an edge $U \leftrightarrow V$ with weight W gives us a strictly shorter distance to V than what we currently know, we update it:

$\text{if } (\text{dist}[U] + W < \text{dist}[V]) \rightarrow \text{dist}[V] = \text{dist}[U] + W$

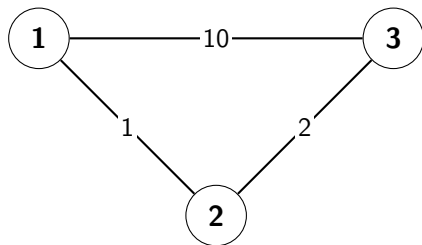
Why BFS Fails: An Example

- Let's find the shortest path from **Node 1** to **Node 3**.



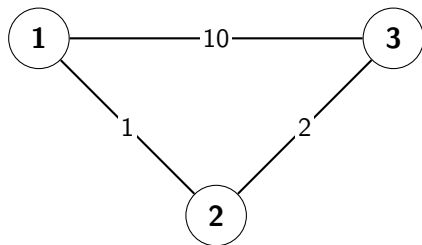
Why BFS Fails: An Example

- Let's find the shortest path from **Node 1** to **Node 3**.
- BFS explores layer-by-layer (by number of edges).



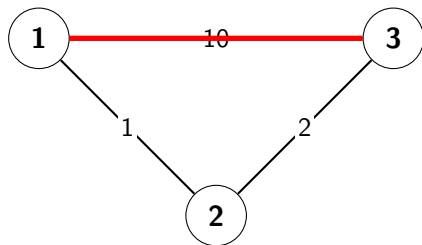
Why BFS Fails: An Example

- Let's find the shortest path from **Node 1** to **Node 3**.
- BFS explores layer-by-layer (by number of edges).
- Layer 1:** BFS visits Node 2 and Node 3.



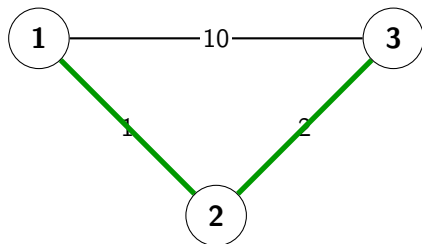
Why BFS Fails: An Example

- Let's find the shortest path from **Node 1** to **Node 3**.
- BFS explores layer-by-layer (by number of edges).
- Layer 1:** BFS visits Node 2 and Node 3.
- It sees a 1-edge path to Node 3 and locks it in (**Cost = 10**).



Why BFS Fails: An Example

- Let's find the shortest path from **Node 1** to **Node 3**.
- BFS explores layer-by-layer (by number of edges).
- Layer 1:** BFS visits Node 2 and Node 3.
- It sees a 1-edge path to Node 3 and locks it in (**Cost = 10**).
- The **actual** shortest path goes through Node 2 (**Cost = 3**). BFS completely misses this because it takes 2 edges!



Bellman-Ford Algorithm

- The algorithm consists of several **phases**.

Bellman-Ford Algorithm

- The algorithm consists of several **phases**.
- Each phase **scans through all edges** of the graph, and tries to **improve** the shortest path along each edge.

Bellman-Ford Algorithm

- The algorithm consists of several **phases**.
- Each phase **scans through all edges** of the graph, and tries to **improve** the shortest path along each edge.
- A simple shortest path can have at most $|V| - 1$ edges. Therefore, it is enough to conduct exactly $|V| - 1$ **phases**.

Bellman-Ford Algorithm

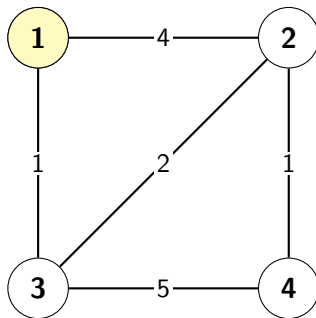
- The algorithm consists of several **phases**.
- Each phase **scans through all edges** of the graph, and tries to **improve** the shortest path along each edge.
- A simple shortest path can have at most $|V| - 1$ edges. Therefore, it is enough to conduct exactly $|V| - 1$ **phases**.
- **Complexity:** $O(|V| \times |E|)$. It works flawlessly with negative edge weights!

Bellman-Ford Pseudocode

Bellman-Ford ($O(V \times E)$)

```
function bellman_ford(V, edges, source) {  
    // Initialize distances to infinity  
    fill dist array with Infinity  
    dist[source] = 0  
  
    // Run V - 1 phases  
    for phase from 1 to V - 1 {  
        for each edge (u, v, weight) in edges {  
            // Try to improve the distance to node v  
            if (dist[u] + weight < dist[v]) {  
                dist[v] = dist[u] + weight  
            }  
        }  
    }  
}
```

Bellman-Ford Dry Run

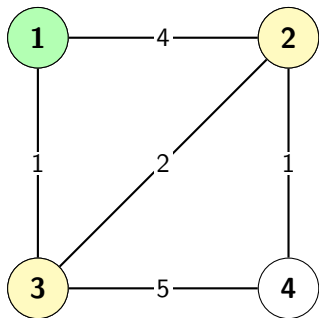


Distances:

$$d = [0, \infty, \infty, \infty]$$

Initialization.

Bellman-Ford Dry Run

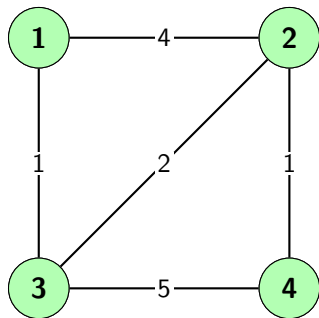


Distances:

$$d = [0, 4, 1, \infty]$$

Partial Phase 1: Scanning edges from 1.

Bellman-Ford Dry Run

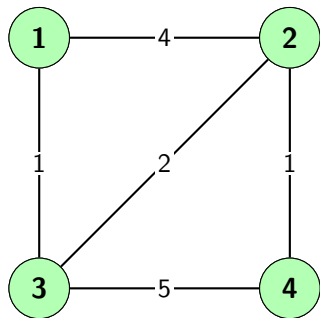


Distances:

$$d = [0, 3, 1, 4]$$

End of Phase 1: All edges scanned.

Bellman-Ford Dry Run



Distances:

$$d = [0, 3, 1, 4]$$

Phase 2: No improvements! Early stop.

The Need for Optimization

- Bellman-Ford works reliably, but checking every edge repeatedly can be slow for large graphs.

The Need for Optimization

- Bellman-Ford works reliably, but checking every edge repeatedly can be slow for large graphs.
- A standard queue won't work anymore because the node with the fewest edges isn't always the shortest path.

The Need for Optimization

- Bellman-Ford works reliably, but checking every edge repeatedly can be slow for large graphs.
- A standard queue won't work anymore because the node with the fewest edges isn't always the shortest path.
- We need a structure that always prioritizes the **shortest weight** path available.

Min-Heap (Priority Queue)

What is it?

A **Min-Heap** also called a **Priority queue** is a tree structure where all the elements in the subtree of a node have key value less than or equal to the node value i.e. the shortest element is always kept at the root of the tree.

- This gives us incredible efficiency:

Min-Heap (Priority Queue)

What is it?

A **Min-Heap** also called a **Priority queue** is a tree structure where all the elements in the subtree of a node have key value less than or equal to the node value i.e. the shortest element is always kept at the root of the tree.

- This gives us incredible efficiency:
 - **Insert a element** $O(\log N)$

Min-Heap (Priority Queue)

What is it?

A **Min-Heap** also called a **Priority queue** is a tree structure where all the elements in the subtree of a node have key value less than or equal to the node value i.e. the shortest element is always kept at the root of the tree.

- This gives us incredible efficiency:
 - **Insert a element** $O(\log N)$
 - **Get the minimum:** $O(1)$

Min-Heap (Priority Queue)

What is it?

A **Min-Heap** also called a **Priority queue** is a tree structure where all the elements in the subtree of a node have key value less than or equal to the node value i.e. the shortest element is always kept at the root of the tree.

- This gives us incredible efficiency:
 - **Insert a element** $O(\log N)$
 - **Get the minimum:** $O(1)$
 - **Remove the minimum:** $O(\log N)$

Min-Heap (Priority Queue)

What is it?

A **Min-Heap** also called a **Priority queue** is a tree structure where all the elements in the subtree of a node have key value less than or equal to the node value i.e. the shortest element is always kept at the root of the tree.

- This gives us incredible efficiency:
 - **Insert a element** $O(\log N)$
 - **Get the minimum:** $O(1)$
 - **Remove the minimum:** $O(\log N)$
- *In C++, this is built right into the STL:*
`std::priority_queue<int, vector<int>, greater<int>>`

Dijkstra's Algorithm

- Dijkstra's is essentially BFS, but we replace the standard Queue with a Priority Queue.

Dijkstra's Algorithm

- Dijkstra's is essentially BFS, but we replace the standard Queue with a Priority Queue.
- Instead of exploring "layer-by-layer", we explore "**shortest weight path first**".

Dijkstra's Algorithm

- Dijkstra's is essentially BFS, but we replace the standard Queue with a Priority Queue.
- Instead of exploring "layer-by-layer", we explore "**shortest weight path first**".

The Strategy

We extract the minimum distance node from the Priority Queue, and we **scan its edges** to try and **improve** its neighbors' distances. If a neighbor gets a cheaper path, we insert it back into the heap!

Complexity: $O(|E| \log |V|)$ — significantly faster than Bellman-Ford!

Dijkstra's Pseudocode

Dijkstra ($O(E \log V)$)

```
function dijkstra(adj, source) {
    fill dist array with Infinity
    dist[source] = 0

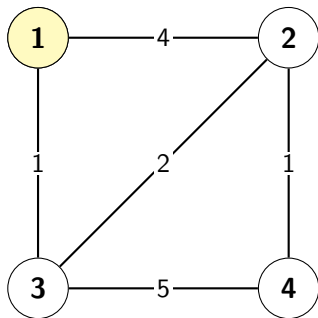
    Create Min-Heap pq
    pq.push((0, source)) // Push (distance, node)

    while pq is not empty {
        cost, u = pq.extract_min()

        // Ignore old data in the heap
        if (cost > dist[u]) continue;

        for each neighbor (v, weight) of u {
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight
                pq.push((dist[v], v))
            }
        }
    }
}
```

Dijkstra Dry Run



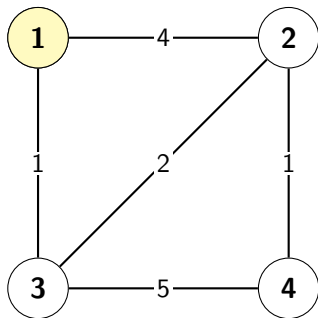
Distances:

$$d = [0, \infty, \infty, \infty]$$

Min-Heap (Cost, Node):

$\{(0, 1)\}$

Dijkstra Dry Run



Distances:

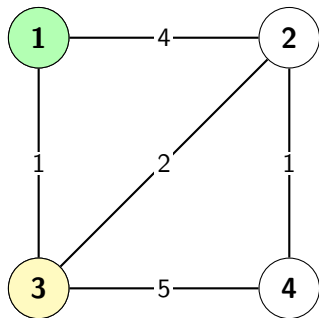
$$d = [0, 4, 1, \infty]$$

Min-Heap (Cost, Node):

$\{(1, 3), (4, 2)\}$

Pop 1. Improve 2 and 3.

Dijkstra Dry Run



Distances:

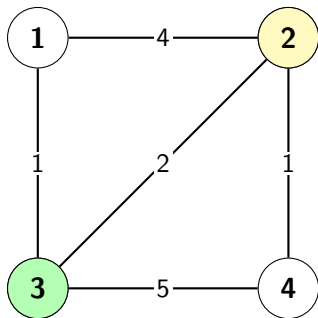
$$d = [0, 3, 1, 6]$$

Min-Heap (Cost, Node):

$\{(3, 2), (4, 2), (6, 4)\}$

Pop 3. Improve 2 ($1 + 2 = 3 < 4$) and 4.

Dijkstra Dry Run



Distances:

$d = [0, 3, 1, 4]$

Min-Heap (Cost, Node):

$\{(4, 2), (4, 4), (6, 4)\}$

Pop 2 (cost 3). Improve 4 ($3 + 1 = 4 < 6$).

The Limitation

- Dijkstra's is a greedy algorithm.

The Limitation

- Dijkstra's is a greedy algorithm.
- The key idea is that once a node is popped from the heap, its shortest path is finalized.

The Limitation

- Dijkstra's is a greedy algorithm.
- The key idea is that once a node is popped from the heap, its shortest path is finalized.

What if there are negative weight edges?

If negative weights exist, the algorithm fails to work and for these cases we must use the Bellman-Ford algorithm.

What is the Diameter of a Tree?

- Recall: A tree is an acyclic, connected graph.

What is the Diameter of a Tree?

- Recall: A tree is an acyclic, connected graph.
- The **Diameter** is the absolute longest possible shortest-path between any two nodes in the tree.

What is the Diameter of a Tree?

- Recall: A tree is an acyclic, connected graph.
- The **Diameter** is the absolute longest possible shortest-path between any two nodes in the tree.
- **Naive Approach:** Run BFS/DFS from every single node to find the maximum possible distance.

What is the Diameter of a Tree?

- Recall: A tree is an acyclic, connected graph.
- The **Diameter** is the absolute longest possible shortest-path between any two nodes in the tree.
- **Naive Approach:** Run BFS/DFS from every single node to find the maximum possible distance.
- **Complexity:** $O(V^2)$ – Too slow!

What is the Diameter of a Tree?

- Recall: A tree is an acyclic, connected graph.
- The **Diameter** is the absolute longest possible shortest-path between any two nodes in the tree.
- **Naive Approach:** Run BFS/DFS from every single node to find the maximum possible distance.
- **Complexity:** $O(V^2)$ – Too slow!
- **Optimal Approach:** We can find the diameter in $O(V)$ time using exactly two DFS traversals.

The 2-DFS Trick

- The algorithm is beautifully simple:

The 2-DFS Trick

- The algorithm is beautifully simple:
- ① **Step 1:** Start a DFS from ANY arbitrary node X .

The 2-DFS Trick

- The algorithm is beautifully simple:
- ① **Step 1:** Start a DFS from ANY arbitrary node X .
- ② **Step 2:** Find the farthest node from X . Let's call it A .

The 2-DFS Trick

- The algorithm is beautifully simple:
- ① **Step 1:** Start a DFS from ANY arbitrary node X .
 - ② **Step 2:** Find the farthest node from X . Let's call it A .
 - ③ **Step 3:** Start a second DFS from A .

The 2-DFS Trick

- The algorithm is beautifully simple:
 - 1 **Step 1:** Start a DFS from ANY arbitrary node X .
 - 2 **Step 2:** Find the farthest node from X . Let's call it A .
 - 3 **Step 3:** Start a second DFS from A .
 - 4 **Step 4:** Find the farthest node from A . Let's call it B .

The 2-DFS Trick

- The algorithm is beautifully simple:
 - 1 **Step 1:** Start a DFS from ANY arbitrary node X .
 - 2 **Step 2:** Find the farthest node from X . Let's call it A .
 - 3 **Step 3:** Start a second DFS from A .
 - 4 **Step 4:** Find the farthest node from A . Let's call it B .

The Result

The path between A and B is guaranteed to be the diameter of the tree.

Tree Diameter Pseudocode

2-DFS Algorithm ($O(V)$)

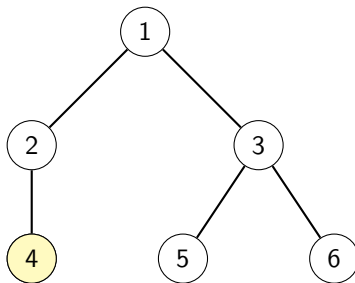
```
// Helper DFS returns (farthest_node, max_distance)
function find_farthest(start_node) {
    run standard DFS/BFS from start_node
    keep track of distances
    return node with maximum distance
}

// Main Algorithm
function tree_diameter(adj) {
    // Step 1 & 2: Find one endpoint of diameter
    node_A, _ = find_farthest(node 1)

    // Step 3 & 4: Find the other endpoint
    node_B, diameter = find_farthest(node_A)

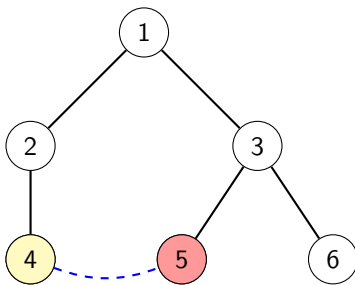
    return diameter
}
```

2-DFS Dry Run



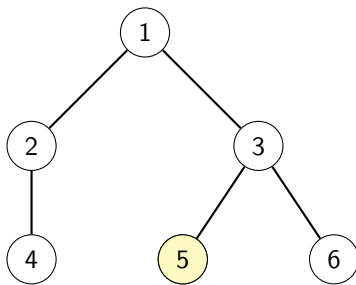
Arbitrary start at Node 4.

2-DFS Dry Run



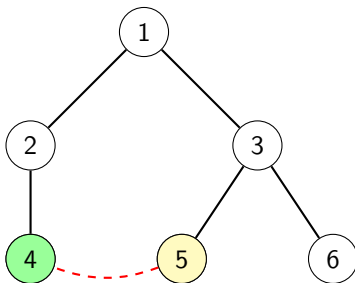
Farthest from 4? It's Node 5 (or 6). This is A.

2-DFS Dry Run



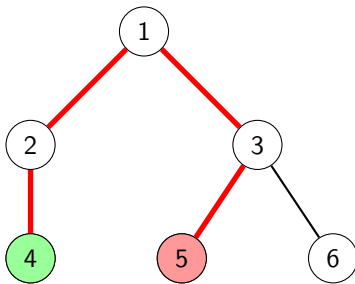
Now, start DFS from A (Node 5).

2-DFS Dry Run



Farthest from 5? It's Node 4. This is B .

2-DFS Dry Run



Diameter is distance between 4 and 5 (Length = 4).